

Blog App API — Role-Based Authentication

Assignment Brief · Backend-only REST API (JSON responses) · Node.js + Express + MongoDB

1. Overview

You will build a **Blog App REST API** — an API-only backend that returns JSON, with no server-rendered views. This assignment extends the pattern you used in the **To-Do List API** (backend - apis/todo_api) by adding **authentication** (signup/login) and **role-based authorization** (who is allowed to do what) on top of a full CRUD resource.

This brief gives you the end-to-end steps and requirements only — **no solution code**. Re-use the same project structure, coding conventions, and JSON response format you used in the To-Do List API.

Learning Objectives

- Structure an Express API using the Model – Routes – Controllers pattern.
- Hash and verify passwords securely with bcrypt.
- Issue and verify stateless JWT tokens for authentication.
- Write Express **middleware** to protect routes and enforce roles.
- Model a one-to-many relationship (User → Blogs) with Mongoose ref.
- Design consistent success/error JSON responses across an entire API.

2. Tech Stack

- **Runtime:** Node.js, ES Modules ("type": "module", same as todo_api)
- **Framework:** Express 5
- **Database:** MongoDB with Mongoose
- **Auth:** bcrypt for password hashing + jsonwebtoken for stateless auth (Passport.js with passport-jwt is an acceptable alternative if you'd rather practice Passport strategies — but since this is a token-based JSON API and not a session-based app, plain bcrypt + JWT is the recommended default)
- **Dev tooling:** dotenv, morgan, nodemon

Note: the To-Do List API's package.json already lists bcrypt and jsonwebtoken as dependencies — reuse that same package.json as your starting point.

3. Project Structure

Follow the exact same folder layout as todo_api, with one addition — a middlewares/ folder:

```
blog_auth_api/
├── config/
│   └── db.js           # mongoose connection (same pattern as todo_api)
├── models/
│   ├── userModel.js    # User schema
│   └── blogModel.js     # Blog schema
├── controllers/
│   ├── authControllers.js # register, login, getProfile
│   └── blogControllers.js # CRUD for blogs
├── middlewares/
│   └── authMiddleware.js # verifies JWT, attaches req.user
```

```

■   ■■■ roleMiddleware.js      # restricts access by role
■■■ routes/
■   ■■■ authRoutes.js
■   ■■■ blogRoutes.js
■■■ .env
■■■ .gitignore
■■■ package.json
■■■ server.js

```

4. Data Models

User

Field	Type / Rules
name	String, required
email	String, required, unique, lowercase, valid email format
password	String, required, min length 6 — store the bcrypt hash , never the plain text
role	String enum: "user" "admin" — default "user"
timestamps	createdAt / updatedAt (schema option, same as todo_api's Task model)

Blog

Field	Type / Rules
title	String, required, min length 3
content	String, required, min length 10
author	ObjectId, ref: "User", required — set from the logged-in user, never from the request body
timestamps	createdAt / updatedAt

5. Environment Variables (.env)

Key	Purpose
PORT	Port the server listens on (e.g. 3004)
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used to sign/verify JWTs — long, random, never committed
JWT_EXPIRES_IN	Token lifetime, e.g. "7d"

6. Response Format Convention

Keep the exact same JSON envelope you used in the To-Do List API for **every** endpoint — success and error alike:

```

{
  "message": "<human readable message>",
  "status": "success" | "failed",
  "data": <object | array | null>
}

```

```
}
```

Use the same status codes you used before: 200 success, 400 bad request / validation error, 401 not authenticated, 403 not authorized (wrong role / not the owner), 404 not found.

7. Step-by-Step Build Plan

Work through these milestones in order. Each one should be a working, testable increment.

Milestone 1 — Project Setup

- 1 Scaffold the folder structure shown in Section 3 inside a new `blog_auth_api/` project.
- 2 Copy `package.json` from `todo_api` and adjust the name; run `npm install`.
- 3 Create `.env` with the variables from Section 5, and a `.gitignore` (copy `todo_api`'s).
- 4 Set up `server.js`: load env, connect DB, mount `express.json()` and `morgan`, mount the two route groups at `/api/auth` and `/api/blogs`, start listening.

Milestone 2 — Database Connection & User Model

- 1 Reuse the `config/db.js` connection pattern from `todo_api` unchanged.
- 2 Create the User schema exactly as specified in Section 4 — do not skip the `role` enum/default.

Milestone 3 — Auth Controller: Register

- 1 Build `POST /api/auth/register`: validate name/email/password are present and meet the rules in Section 4.
- 2 Check no existing user has that email; if one does, respond 400 with a clear message.
- 3 Hash the password with `bcrypt` before saving — decide and document your salt rounds (10 is a reasonable default).
- 4 Create the user with role always forced to "user" from the server side, regardless of anything sent in the request body.
- 5 Respond with the created user's public fields only — never return the password hash in any response, ever.

Milestone 4 — Auth Controller: Login

- 1 Build `POST /api/auth/login`: find the user by email; if not found, respond 400/401 without revealing whether the email or the password was wrong (just "Invalid credentials").
- 2 Compare the submitted password against the stored hash with `bcrypt`.
- 3 On success, sign a JWT containing the user's id and role, using `JWT_SECRET` and `JWT_EXPIRES_IN`, and return it in the response data.

Milestone 5 — Auth Middleware

- 1 Write `middlewares/authMiddleware.js`: read the token from the `Authorization: Bearer <token>` header.
- 2 If missing or invalid/expired, respond 401 and stop the request — do not call `next()`.
- 3 If valid, fetch the user (or trust the decoded payload) and attach it to `req.user`, then call `next()`.

Milestone 6 — Role Middleware

- 1 Write `middlewares/roleMiddleware.js` as a small factory function that takes a list of allowed roles and returns a middleware.
- 2 It should run **after** `authMiddleware`, read `req.user.role`, and respond 403 if the role isn't in the allowed list, otherwise call `next()`.

Milestone 7 — Blog Model & Controller (CRUD)

- 1 Create the Blog schema exactly as specified in Section 4.
- 2 Build the five controller functions in `blogControllers.js`, following the same try/catch + validation + response-envelope style as `taskControllers.js`: GET all, GET one, CREATE, UPDATE, DELETE.
- 3 On CREATE, set `author` from `req.user.id` — never trust an `author id` sent in the request body.
- 4 On UPDATE/DELETE, first fetch the blog; if it doesn't exist, 404. Then apply the ownership rule from Section 9 before modifying it.

Milestone 8 — Wire Up Routes & Protection

- 1 In `authRoutes.js`: `register` and `login` are public; add one protected route, e.g. `GET /api/auth/me`, that returns the logged-in user's own profile.
- 2 In `blogRoutes.js`: apply `authMiddleware` to every route except the two public GET routes (list all, get one) per the table in Section 9.

Milestone 9 — Enforce Ownership + Role Rules

- 1 Inside the UPDATE and DELETE controllers, allow the action only if `req.user.id === blog.author` **or** `req.user.role === "admin"`.
- 2 Anything else should return 403 with a message like "Not authorized to modify this blog".

Milestone 10 — Manual Testing

- 1 Using Postman/Thunder Client, walk through: register → login → copy token → create a blog → try updating it as the same user (should succeed) → try updating it as a different logged-in user (should 403) → try again as an admin user (should succeed) → delete it.
- 2 Also test the failure paths: missing token, expired/garbled token, duplicate email on register, wrong password on login, invalid blog id on update.

8. API Endpoints

Method & Path	Access	Description
POST <code>/api/auth/register</code>	Public	Create a new account (role always defaults to "user")
POST <code>/api/auth/login</code>	Public	Authenticate and receive a JWT
GET <code>/api/auth/me</code>	Logged-in user	Return the current user's own profile
GET <code>/api/blogs</code>	Public	List all blogs
GET <code>/api/blogs/:id</code>	Public	Get a single blog by id
POST <code>/api/blogs</code>	Logged-in user	Create a blog (author = current user)
PUT <code>/api/blogs/:id</code>	Owner or admin	Update a blog

Method & Path	Access	Description
DELETE /api/blogs/:id	Owner or admin	Delete a blog

9. Role-Based Access Rules

Role	Can do
user (default)	Register/login; read all blogs; create blogs; update/delete only their own blogs; view their own profile
admin	Everything a user can do, plus update/delete any user's blog

There is no public signup path to become an admin — for this assignment it's enough to create an admin by manually setting role: "admin" on a user document directly in MongoDB (e.g. via Compass or the shell) after registering normally.

10. Bonus Challenges (optional)

- Add GET /api/blogs/mine — return only the logged-in user's own blogs.
- Add pagination (?page=&limit;=) and simple title search (?search=) to the list-blogs endpoint.
- Add an admin-only GET /api/users endpoint to list all registered users.
- Add a PUT /api/auth/me endpoint so a user can update their own name/password.
- Rate-limit the login endpoint to slow down brute-force attempts.

11. Submission Checklist

- Folder structure matches Section 3 (config / models / controllers / middlewares / routes / server.js).
- Passwords are never stored or returned in plain text.
- JWT is required (via authMiddleware) on every route that should be protected, per Section 8.
- Ownership + role checks match Section 9 exactly.
- Every response — success and error — follows the envelope in Section 6.
- .env is git-ignored; a .env.example (no real secrets) is committed instead.
- README with setup steps and a short list of all endpoints.